Algorithms Task

Diagonal Difference

Analysis & Time Complexity

Algorithm 1: Non-Recursive

The non-recursive algorithm uses a single for-loop to traverse the matrix once, accumulating both diagonal sums simultaneously. This is a straightforward iterative approach with minimal memory usage.

Step-by-step Analysis:

Step	Operation	Cost	Times Executed
1	**Initialize left_to_right = 0, right_to_left = 0**	O(1)	1
2	**Loop: for i = 0 to n-1**	O(1) per iteration	n
3	**left_to_right += matrix[i][i]**	O(1)	n
4	**right_to_left += matrix[i][n-i-1]**	O(1)	n
5	**return fabs(left_to_right - right_to_left)**	O(1)	1

Complexity Summary:

Metric	Value	Reason
Time Complexity	**O(n)**	**Single loop over n rows**
Space Complexity	**O(1)**	**Only 2 integer variables used**
Best Case	**O(n)**	**Always iterates n times**
Worst Case	**O(n)**	**Always iterates n times**

Algorithm 2: Recursive

The recursive algorithm replaces the loop with recursive function calls. Each call processes one row and passes updated sums to the next call until the base case (i = n) is reached.

How it works:

- The main program starts by reading the integer n, which represents the size of the square matrix.
- The recursive function solve(matrix, n, 0, 0, 0) is called with the initial index i = 0 and both primarySum and secondarySum initialized to 0.
- In each recursive call, the element matrix[i][i] is added to primarySum (primary diagonal) and matrix[i][n−1−i] is added to secondarySum (secondary diagonal).
- The function then calls itself with the next index i+1 while passing the updated sums.
- The recursion continues until the base case i == n is reached, then returns |primarySum − secondarySum|.

Step-by-step Analysis:

Step	Operation	Cost	Times Executed
1	**Initial call: solve(matrix, n, 0, 0, 0)**	O(1)	1
2	**Check base case: if i == n**	O(1)	n + 1
3	**primarySum += matrix[i][i]**	O(1)	n
4	**secondarySum += matrix[i][n-1-i]**	O(1)	n
5	**Recursive call: solve(matrix, n, i+1, p, s)**	O(1)	n
6	**Base case: return |primarySum - secondarySum|**	O(1)	1

Complexity Summary:

Metric	Value	Reason
Time Complexity	**O(n)**	**n recursive calls, each O(1)**
Space Complexity	**O(n)**	**n stack frames on the call stack**
Best Case	**O(n)**	**Always makes n recursive calls**
Worst Case	**O(n)**	**Always makes n recursive calls**

Comparison Between Both Algorithms

Criteria	Non-Recursive	Recursive

Method	Uses iteration with loops	Uses recursion with function calls
Time Complexity	**O(n)**	**O(n)**
Space Complexity	**O(1) — constant space**	**O(n) — call stack**
Performance	Faster — no recursive overhead	Slightly slower — recursive call overhead
Code Structure	Simple and direct	More conceptual but harder to trace
Risk	**None**	Stack overflow for very large n
Preferred	**Yes**	**No**

Conclusion:

The non-recursive solution calculates the sums of the two diagonals using a loop that iterates through the matrix once. This approach is memory efficient because it uses constant space O(1). On the other hand, the recursive solution computes the diagonal sums by repeatedly calling a function for each matrix index. Although both approaches have the same time complexity O(n), the recursive approach consumes more memory due to the recursive call stack O(n). Therefore, the non-recursive approach is generally more efficient in terms of memory usage and performance.